

JWT — JSON Web Tokens

Key Concepts

- **JWT** — compact, self-contained token used to authenticate users across requests
- **Stateless** — server stores nothing; all info is inside the token
- **Base64URL encoded** — not encrypted; anyone can decode the payload
- **Signature** — HMAC-SHA256 hash of header + payload using a secret key; prevents tampering
- **Claims** — key-value pairs inside the payload (userId, role, exp, etc.)

Structure

```
Header.Payload.Signature
xxxxx.yyyyy.zzzzz
```

Part	Content	Example
Header	Algorithm + token type	{ "alg": "HS256", "typ": "JWT" }
Payload	Claims (user data + metadata)	{ "userId": 42, "role": "admin", "exp": 1716000000 }
Signature	HMAC of header+payload	Prevents tampering

Common Claims

Claim	Meaning
sub	Subject (user ID)
exp	Expiry timestamp
iat	Issued at
iss	Issuer
role	Custom — user's role

How It Works

1. User sends username + password
2. Server verifies credentials
3. Server creates JWT, signs it with a secret key
4. Server returns JWT to client
5. Client stores it (memory or localStorage)
6. Every request: client sends `Authorization: Bearer <token>`
7. Server verifies signature — if valid, trusts the claims inside (no DB lookup)

JWT vs Session-Based Auth

	JWT (Stateless)	Session (Stateful)
Storage	Client holds token	Server holds session data
Scalability	Easy — no shared state	Harder — need sticky sessions or shared store
Revocation	Hard — valid until expiry	Easy — delete session from store
Request size	Larger (token every request)	Smaller (just a session ID)

Security Rules

- Never put sensitive data (passwords, SSNs) in the payload — it's only encoded, not encrypted
- Always use short expiry (15 min access token) + refresh tokens
- Revocation options: token blacklist in Redis (reintroduces state) or wait out expiry
- Refresh token pattern: short-lived access token + long-lived refresh token (7 days)

Industry Examples

- **Netflix** — JWTs passed between microservices; streaming service verifies without calling auth service
- **Google APIs** — JWT-based identity tokens in OpenID Connect
- **Auth0 / Okta** — issue JWTs as access tokens for API authorization

Interview Questions

Q: What is a JWT and why use it over sessions?

Self-contained token with header, payload, signature. Server signs it on login; client sends it on every request. Server verifies signature — no DB lookup needed. Scales horizontally because no shared session state.

Q: The payload is Base64 encoded — what does that mean?

Anyone can decode it — it's not encrypted. Never store passwords or sensitive PII in the payload. Only store non-sensitive identifiers like user ID and role.

Q: How do you handle JWT revocation?

Short expiry (15 min) limits the damage window. For immediate revocation, maintain a blacklist in Redis — but this reintroduces state. Common pattern: short-lived access token + longer-lived refresh token.

Q: What's the difference between access token and refresh token?

Access token is short-lived (minutes), sent on every API request. Refresh token is long-lived (days/weeks), stored securely, only used to get a new access token when the old one expires — without requiring re-login.